

CV Sorting System Documentation (Complete Guide)

This document serves as a comprehensive guide to the architecture, features, user workflows, and exhaustive file-level structure of the CV Sorting application. It is designed to provide future developers and administrators with a clear understanding of how the system operates from both the user and admin perspectives.

1. Authentication & Role-Based Access Control (RBAC)

1.1 Login & Forgot Password

- **Login** (`index.php`, `check_user.php`): Authenticates users against the `users` table and sets PHP session variables (Username, Role, and granular Permissions JSON).
- **Forgot Password** (`api/forgot_password_api.php`): Allows users to request a password reset via their registered email. System generates a secure token and sends an automated reset email.

1.2 Role Hierarchy & Permissions Engine (`config/auth.php`)

- **Roles**: Root Admin (Joy Ballav - 097727, bypasses all checks), Super-Admin, Admin, User/Sub-Admin.
 - **has_permission(\$key)**: Core security function returning `true` or `false` based on the user's specific feature flags.
 - **Real-time Sync**: `syncAuthPermissions()` fetches `api/auth_check_api.php` every 3 seconds to instantly reflect permission changes on the frontend UI without a page reload.
-

2. Dashboard & Navigation

2.1 Modern React Sidebar (`includes/sidebar_modern.php`)

- **State Management**: Menu visibility is strictly driven by the user's permissions passed from the PHP session into React state (`permissionsLoaded` event).
- **Theme Settings**: Features built-in Dark Mode and UI Zoom functionality.

2.2 Task View Modes ("My Task" vs. "All Task")

- **My Task**: Displays tasks created by the logged-in user **OR** tasks explicitly shared with them via the `allowed_viewers` column.
 - **All Task**: Displays all tasks system-wide (requires `view_all_task` permission). Test tasks are hidden from everyone except their creators and root admins.
-

3. Core Feature: Task Management (`tasks.php`, `js/script.js`)

3.1 Creating Tasks (`api/insert_job.php`)

Users create tasks specifying BDJobs/Manual sources, Job Title, JD ID, and AI Prompts. Backend enforces Daily and Monthly limits before insertion.

3.2 Task Edit Limit & Lifecycle (`api/get_jobs.php`)

Newly created tasks are in a `created` status for a grace period (default 5 minutes) where they can be edited. After expiry, tasks lock and transition to `pending` (awaiting RPA bot) or `downloaded` (manual).

3.3 Task Sharing & Modification

- **Task Share**: Creators or admins can add Employee IDs to the `allowed_viewers` list via the frontend modal. This allows other standard users to view the task in their "My Task" tab.
 - **Task Delete**: Controlled by `delete_task` permission. Soft or hard deletes the task record.
 - **Update Task Status**: Users can manually push tasks through states (e.g., `completed`, `active`) via `api/update_task_status.php`.
-

4. Candidate Dashboard & Operations

When clicking on a Task's CV count, users enter the Candidate Dashboard.

4.1 JD and CV File Handling

- **JD Download:** Users can view or download the original Job Description PDF/Word document uploaded during task creation (`api/view_jd.php`).
- **CV Download:** Users can view or download specific candidate resumes uploaded against a task (`api/view_cv.php`).

4.2 Candidate Evaluation

- **AI Screening:** Candidates are evaluated via AI and given a `match_percentage`.
 - **Export to Excel:** Users can download the entire filtered candidate list into a `.csv` or `.xlsx` format for external reporting (`api/export_candidates.php`).
-

5. Automated Mail & Notifications

5.1 Concern Person Mail Share (`api/send_filtered_mail.php`)

When sorting is complete, users can select a subset of candidates and trigger an automated email directly to the internal "Concern Person" (Hiring Manager). The email includes a breakdown of the candidates and a secure link to view their profiles.

5.2 Task Completion Mail (`api/send_task_mail.php`)

Upon a task transitioning to the `completed` status, the system automatically dispatches an email to the task creator notifying them that AI screening has finished.

6. System Administration (Admin End)

6.1 System Users (`api/user_api.php`)

- **Role Hierarchy Guard:** Admins can only view and manage accounts at or below their own tier.
- **Permissions Modal:** Granular UI to toggle exactly which buttons and views a user can access. Backend strictly verifies that an admin cannot delegate a permission they don't possess themselves.

6.2 DB Control (`api/db_control_api.php`)

A critical dashboard for root/super-admins to execute structural optimizations, table truncations, or system resets.

6.3 Master Data Modals

- **Manage Sources:** Add/Remove CV sources (e.g., BDJobs, LinkedIn, Manual).
- **Manage Statuses:** Define the lifecycle states a task or candidate can be in (`api/status_api.php`).
- **MGI Employees Sync:** Pulls the latest employee roster to auto-populate internal Dropdowns and validation checks.

6.4 Server Allocation (`api/update_server_config.php`)

Allows root admins to dynamically map which external LLM inference server (e.g., Gemini, OpenAI, Local LLM) handles the AI screening traffic for specific task types to load balance the system.

6.5 Web Config & RPA Config (`api/rpa_config_api.php`)

- **RPA Config:** Controls the Robotic Process Automation bots. Admins can tweak bot sleep times, chunk sizes, and API endpoint hooks.
- **Task Limits:** Admins set daily and monthly maximum limits for task generation to prevent system abuse (`api/task_limit_api.php`).

6.6 User Activity (`api/user_activity_api.php`)

An audit log view tracking when users log in, their IP addresses, and their session actions to maintain system accountability.

7. API Dictionary (/api/ Directory)

The backend is completely modularized. Below is the purpose of every key API in the system:

7.1 Authentication & Core System APIs

- **auth_check_api.php:** Polled every 3 seconds by the frontend to validate session validity and fetch the user's latest granular permissions.
- **forgot_password_api.php:** Handles password reset requests, generates tokens, and sends recovery emails.
- **user_api.php:** The core user management router. Handles fetching the user list, updating inline roles, and modifying granular

permissions.

- **user_activity_api.php**: Returns audit logs of user actions (logins, config changes) for the admin dashboard.

7.2 Task Creation & Dashboard APIs

- **insert_job.php**: Handles the creation of new CV Sorting tasks. Enforces daily/monthly limits and saves JD files/prompts.
- **get_jobs.php**: Fetches the massive task list for the dashboard. Calculates the 5-minute edit lifecycle timer dynamically.
- **job_api.php**: A versatile router for task actions (e.g., Deleting tasks, explicitly updating `allowed_viewers` for Task Sharing).
- **update_task_status.php**: Updates a task's state (e.g., moving from `pending` to `active` or `completed`).
- **get_job_task_data.php**: Fetches the specific details (Prompt, Source, Status) of a parent task when entering Candidate View.
- **download_task_zip.php**: Bundles all Candidate CVs and the Job Description into a downloadable `.zip` file.

7.3 Candidate Management APIs

- **get_candidates.php**: Fetches the grid list of candidates for a specific task. Used extensively inside the candidate dashboard.
- **insert_candidate.php**: (Used by RPA Bot & Manual Upload) Inserts a new parsed candidate into the DB, logging their AI match percentage and details.
- **export_candidates.php**: Converts the candidate dashboard grid data into a downloadable Excel/CSV file.
- **view_cv.php / view_jd.php**: Secure file servers that fetch and display PDF/Word files from the local filesystem to the user.
- **update_shortlisted.php**: Toggles a specific candidate's shortlisted state (Yes/No).

7.4 Automated RPA Bot Integrations (Machine-to-Machine)

- **rpa_config_api.php**: The central nervous system for bot settings. The RPA bots poll this to know their sleep timers and chunk processing sizes.
- **trigger_screening.php**: A manual trigger hook to kick off backend automated screening pipelines.
- **update_cv_count.php / update_profile_count.php**: Pinged constantly by the RPA bots to update the user dashboard in real-time on how many CVs it has scraped.
- **n8n_api.php**: Webhook endpoints for integrations with n8n automated workflow tools.

7.5 Admin Configuration APIs

- **db_control_api.php**: Allows root-level admins to perform structural database maintenance and optimize tables.
- **employee_api.php**: Synchronizes and lists internal MGI employee data to populate assignment dropdowns.
- **prompt_api.php**: CRUD operations for the AI prompt templates used during task creation.
- **status_api.php / source_api.php**: CRUD operations for the dropdown master data (Status lists, Source lists).
- **task_limit_api.php**: Configures the strict monthly and daily task caps.
- **column_templates.php**: Saves user-specific column layouts and preferences for their dashboard grids.
- **update_server_config.php / get_server_config.php**: Directs AI traffic by mapping tasks to specific LLM servers (e.g., Local, OpenAI, Gemini).

8. Exhaustive File-Level Breakdown & Codebase Map

This section documents every file inside the project directory, explaining its connection to the overall system and explicitly flagging unnecessary files for deletion.

8.1 Root Directory (/)

Essential Core Files

- **login.php**: The main login UI. Contains the HTML/CSS for the login form and handles session initialization upon a successful login.
- **check_user.php**: The backend endpoint that `login.php` POSTs to. It queries the `users` table, verifies passwords, checks for `status = 'active'`, and sets `$_SESSION` variables.
- **index.php**: The primary "Home" dashboard. Displays aggregate system metrics, recent activities, and provides the main React sidebar navigation.
- **tasks.php**: The core "Task Management" dashboard. Displays the grid of all Job/Task entries (filtered by "My Task" / "All Task"), where users create tasks and open candidate views.
- **schema.sql**: Extremely important structural file. Contains the master database schema, table definitions, and triggers.
- **download.php**: A utility endpoint proxying file downloads securely (e.g., zip files or Excel sheets).

● Unnecessary / Debugging Files (Safe to Delete)

- **index.php.bak / index_test.php**: Old backups and test versions of the home page.
- **test_dashboard.php**: Scaffolded test file.
- **check_bom.php, check_db.php, check_job_schema.php, check_jobs_list.php, check_statuses.php**: Temporary

scripts used to verify database connectivity.

- **debug_data.php, debug_jd.php, debug_titles.php, debug_user.php, debug_users.php:** Scripts to var_dump/print JSON data during development.
- **fix_candidate_titles.php, fix_status.php, fix_status_order.php:** One-off database migration/fix scripts. Once run, they serve no future purpose.
- **init_root_admin.php:** Script used to seed the initial root admin account in a fresh database.
- **migrate_remember_token.php:** Migration script for setting up "Remember Me" tokens.
- **sync_counts.php, update_db.php, update_serper_key.php, tmp_describe.php:** Utility/scratch scripts used for backend structural tweaks.

8.2 Configuration (/config/)

Essential Files

- **db.php:** Establishes the global `$conn` MySQLi database connection used by every other PHP file.
- **auth.php:** The core Permissions Engine. Contains `has_permission()`, definition of the Root Admin ID, and enforces RBAC hierarchy rules.
- **mail_sender.php:** A centralized wrapper class (PHPMailer) that connects to the SMTP server.
- **cacert.pem:** SSL Certificate bundle used by cURL within PHP to make secure HTTPS requests to external APIs.

8.3 Includes & Modals (/includes/)

Essential Files

- **sidebar_modern.php:** The React-powered navigation menu. It reads PHP session state directly on load to immediately render menu items without visual flickering.
- **modals.php:** A massive file containing the HTML markup for almost every popup dialog in the system (e.g., Create Task, Edit User, Config settings). Keeps main files clean.
- **path_helper.php:** Utility for dynamically resolving absolute vs relative paths across different server environments.

8.4 Views (/view/)

Essential Files

- **dashboard.php:** The main Candidate View UI. Renders the grid of parsed candidates, AI matching percentages, and CV viewing features.
- **reset_password.php:** The UI page a user lands on when recovering their password via email.

● Unnecessary / Legacy Files

- **dashboard_modern_old.php, dashboard_test.php, dashboard_v1.php:** Legacy backups of older candidate dashboard layouts.
- **entry_debug.txt:** Junk log file.

8.5 JavaScript (/js/)

Essential Files

- **script.js:** The monolithic frontend controller (~5,000+ lines). Handles modal toggling, fetching data from `/api/` endpoints, rendering `window.allJobsData` into HTML tables, real-time search/filtering (`applyJobFilters()`), and enforcing view modes.

● Unnecessary Files

- **script_test.js:** A development backup/test version of the main script.